TeamOps Viva Preparation

Complete Question Bank, Model Answers, Tricky Follow-Ups, and Examiner Strategy

Name: _____ | Roll No: _____ | Section: _____

Updated: _____

TeamOps Viva Preparation

This document is written so you can explain TeamOps like you understand the full system end to end. Do not memorize every word. Learn the ideas, then answer naturally.

30-Second Pitch

TeamOps is a full-stack real-time collaborative project management platform. It lets users create workspaces, invite members, manage roles, create Kanban boards, add columns and cards, assign tasks, set due dates, comment, receive notifications, and track activity. The frontend is a standalone HTML/CSS/JavaScript app served with Vite. The backend is Node.js and Express with MongoDB through Mongoose. Authentication uses JWT, passwords are hashed using bcrypt, and real-time updates are handled by Socket.io rooms for boards, users, and workspaces.

2-Minute Architecture Answer

The system has three main parts: frontend, backend API, and database. The frontend handles views, forms, dashboard rendering, drag-and-drop, local session storage, and Socket.io client events. It communicates with the backend through REST APIs using a JWT Bearer token. The backend validates the JWT, checks workspace roles, performs database operations through Mongoose, and returns formatted JSON. For real-time actions, the backend also emits Socket.io events. For example, when a card is moved, the backend checks permission, updates the card position in MongoDB, creates an activity log, optionally sends a notification, and emits `card:moved` to the `board:<boardId>` room so connected clients refresh instantly.

Must-Know Facts

Topic	Answer
Project name	TeamOps
Project type	Full-stack real-time project management app
Frontend	Standalone HTML/CSS/vanilla JS in `teamops-frontend/index.html` and `styles.css`

Topic	Answer
Backend	Node.js, Express 5, Socket.io
Database	MongoDB
ODM	Mongoose
Auth	JWT, bcrypt, OTP verification, Google sign-in
Realtime	Socket.io
Main entities	User, Workspace, Board, Column, Card, Comment, Activity, Notification
Roles	owner, admin, member, viewer
Default columns	To Do, In Progress, In Review, Done
Seed users	zainab, ahmed, zunairah, hassan at `teamops.dev`, password `123456`
Important events	`card:moved`, `activity:new`, `notification:new`, `workspace:presence`

Basic Questions

What is TeamOps?

TeamOps is a real-time collaborative project management platform. It allows teams to manage tasks using workspaces, boards, columns, cards, comments, notifications, activity logs, and analytics.

Why did you build it?

I built it to solve the problem of scattered project coordination. In team projects, tasks often get lost in chat messages or spreadsheets. TeamOps centralizes ownership, deadlines, progress, and communication in one workspace.

What makes it different from a simple CRUD app?

It includes real-time synchronization, role-based permissions, JWT authentication, email verification, Google login, comments, notifications, activity logs, analytics, member roles, invite workflows, and drag-and-drop board movement.

What is the tech stack?

Frontend is HTML, CSS, and vanilla JavaScript served by Vite. Backend is Node.js with Express 5. Database is MongoDB through Mongoose. Real-time communication uses Socket.io. Authentication uses JWT and bcrypt. Google sign-in uses Google Auth Library.

Is your frontend React?

No. The active frontend is a standalone HTML/CSS/JavaScript app. There are some legacy React dependencies in `package.json`, but the actual app logic is in `teamops-frontend/index.html` and

styling is in `teamops-frontend/styles.css`.

Why did you use vanilla JavaScript instead of React?

For this version, I wanted full control and a simple static deployment model. The app is complex enough to demonstrate frontend state management, API integration, DOM rendering, and real-time updates without depending on a component framework. A future improvement could be modularizing the frontend or migrating to a framework if maintainability becomes harder.

What database did you use?

MongoDB, accessed through Mongoose schemas and models.

Why MongoDB?

The project has document-friendly data such as workspace members, board structures, comments, notifications, and activity logs. MongoDB is flexible for iterative development, while Mongoose still gives schema validation, references, indexes, and structured queries.

What is the role of Express?

Express handles HTTP routing, middleware, JSON request parsing, protected routes, and REST API endpoints for auth, workspaces, boards, cards, activities, notifications, and user profile operations.

What is the role of Socket.io?

Socket.io provides real-time bidirectional communication. It lets the server push updates to connected clients, such as card movement, new comments, new activities, notifications, and presence counts.

Feature Questions

What are the main features?

- Register/login with JWT.
- Email OTP verification and password reset.
- Google sign-in.
- Create and join workspaces.
- Invite members with codes and email tokens.
- Owner/admin/member/viewer roles.
- Multiple boards per workspace.
- Custom columns.
- Cards with assignee, priority, due date, description, and position.
- Drag-and-drop card movement.
- Card and board comments.
- Mentions and tagged card references.
- Live activity feed.
- Notifications with unread and important status.
- Board analytics.

- Profile and assigned work page.

Explain workspace creation.

When a user creates a workspace, the backend validates the name, normalizes the tech stack, generates a unique invite code, creates the workspace with the user as owner, creates a default board called `Sprint 4`, and creates default columns.

Explain board loading.

The frontend reads the active board id from localStorage, joins the Socket.io board room, calls `GET /api/boards/:boardId`, receives board metadata, columns, cards, and activity logs, stores it in frontend state, then renders the Kanban board.

Explain card creation.

The user opens a modal, enters card details, selects a column, assignee, priority, and due date. The frontend sends a POST request to `/api/boards/:boardId/cards`. The backend checks role permission, verifies the column belongs to the board, validates assignee and due date, calculates the next position, creates the card, logs activity, notifies the assignee if necessary, emits `card:created`, and returns the formatted card.

Explain card movement.

The frontend uses drag-and-drop. On drop, it sends a PATCH request to `/api/boards/:boardId/cards/:cardId/move` with destination column and index. The backend checks that both source and destination columns belong to the same board, recalculates positions, saves the card, creates activity, optionally notifies the assignee, and emits `card:moved`.

Explain notifications.

Notifications are stored in the `Notification` collection. They belong to a user and include message, read status, important status, and creation time. The backend emits `notification:new` to the private user room, and the frontend updates the notification inbox and badge.

Explain activity logs.

Activities are records of important actions such as creating cards, moving cards, inviting users, changing roles, and updating accounts. They help users understand what happened on a board or workspace.

Explain audit log versus activity feed.

Activity feed is for collaboration events. Audit-style filtering focuses on governance-sensitive events like workspace changes, invites, member roles, password/account actions, and deletions. Owners/admins can see broader governance activity; other users see limited governance activity.

Explain analytics.

The analytics endpoint loads board columns and cards, then calculates cards per column, cards completed this week, cards per assignee, and overdue card count. The frontend renders those values in the analytics dashboard.

Authentication Questions

What is JWT?

JWT means JSON Web Token. It is a signed token that stores claims such as `userId`. The client sends it with protected API requests. The server verifies the signature and extracts the user id.

Why use JWT?

JWT works well for stateless API authentication. The backend does not need a server-side session table for normal requests. Each request carries the token, and the server verifies it using `JWT_SECRET`.

Where do you store the token?

The frontend stores it in `localStorage` under `teamops_token`.

Is localStorage perfectly secure?

No. It is convenient, but if an XSS vulnerability exists, malicious JavaScript could read localStorage. A more secure production improvement would be HTTP-only secure cookies, combined with CSRF protection if needed.

How are passwords stored?

Passwords are not stored directly. The backend hashes them using bcrypt and stores only `passwordHash`.

Why bcrypt?

bcrypt is intentionally slow and salted, which makes brute-force attacks harder if password hashes are leaked.

What is email verification?

After registration, the backend creates a six-digit OTP token with an expiry time. The user must provide the code before the account becomes verified and before normal login is allowed.

Why verify email?

It prevents fake or mistyped emails and makes password reset/invite workflows more reliable.

How does forgot password work?

The user submits email. If the email exists, the backend creates an OTP reset token and sends it. The response is generic to avoid revealing whether the email exists. Then the user submits email, token, and new password. The backend validates the token and updates the password hash.

How does Google login work?

The frontend obtains a Google credential. The backend verifies the ID token using the configured Google client id. If valid, it finds or creates the user, marks them verified, stores Google id and avatar if available, and returns a TeamOps JWT.

What happens if a Google user tries password login?

If the account uses Google and has no password hash, the backend rejects password login and tells the user to continue with Google.

Authorization and Roles

What is RBAC?

RBAC means role-based access control. Users get permissions based on their assigned role in a workspace.

What roles exist?

Owner, admin, member, and viewer.

What can owners do?

Owners have full control. They can manage workspace settings, boards, columns, members, roles, invites, and delete the workspace.

What can admins do?

Admins can manage boards, columns, members, invites, and settings, but they cannot remove or demote the owner.

What can members do?

Members can collaborate on boards: create, edit, delete, move cards, comment, and participate in work.

What can viewers do?

Viewers can read board data and comments. They have restricted editing permissions.

How does the backend enforce roles?

Protected routes use JWT auth first. Then `requireRole` or controller helpers resolve the workspace and check whether `req.userId` exists in `workspace.members` with a suitable role.

Why check permissions on the backend if the frontend hides buttons?

Frontend checks are only for user experience. A user can still send requests manually. Backend checks are mandatory for security.

What happens if a member tries to create a column?

The backend rejects it because column management requires owner or admin role.

What happens if a viewer tries to create a card?

The backend rejects it because card creation requires owner, admin, or member.

Database Questions

What is Mongoose?

Mongoose is an ODM for MongoDB. It lets me define schemas, models, validation, references, defaults, and query helpers in JavaScript.

Why are formatters used?

The backend uses formatter functions like `formatUser`, `formatCard`, and `formatWorkspace` to control API output, convert MongoDB `_id` values to `id`, normalize dates, and expose clean JSON keys.

What is the User schema?

It stores name, email, password hash, avatar URL, Google id, auth provider, verification status, and notification preferences.

What is the Workspace schema?

It stores workspace name, description, tech stack, owner, invite code, and members. Each member has a user reference and role.

What is the Board schema?

It stores workspace id, board name, color, and timestamps.

What is the Column schema?

It stores workspace reference, board reference, title, and position.

What is the Card schema?

It stores column, title, description, priority, assignee, due date, position, created timestamp, and updated timestamp.

Why store card position?

Position lets the board preserve card ordering inside a column after refresh and across users.

Why not embed cards inside boards?

Referencing cards separately makes updates, comments, analytics, and movement easier to query and update. If all cards were deeply embedded, large board documents could become harder to update and scale.

Why are there both `workspace` and `workspaceId` fields in Activity?

The model supports both older and newer naming patterns so activity formatting remains compatible. It is partly for backward compatibility while the app evolved.

What is a sparse unique index?

For `googleId`, sparse unique means MongoDB enforces uniqueness only when the field exists. That allows normal email users to have no Google id.

Socket.io Questions

What is a Socket.io room?

A room is a server-side grouping of sockets. TeamOps uses rooms like `board:<boardId>`, `user:<userId>`, and `workspace:<workspaceId>` so events are sent only to relevant clients.

Why not broadcast every event to everyone?

Broadcasting to everyone would leak data and waste resources. Board updates should only go to users viewing that board, notifications should only go to the target user, and presence should only go to the workspace.

How does a user join a board room?

The frontend emits `board:join` or `join:board` with the board id. The socket server joins `board:<boardId>`.

How are private notifications sent?

When a socket connects with a valid JWT, the server joins `user:<userId>`. Notifications are emitted to that private room.

How is presence calculated?

The socket server maintains a `Map` of workspace ids to user connection counts. When a socket joins or leaves a workspace, the server updates the count and emits `workspace:presence`.

What happens if a user has multiple tabs open?

The presence map tracks counts per user id, so multiple socket connections can be represented without immediately removing the user until their last relevant connection leaves.

What is the limitation of the current Socket.io setup?

It works for a single backend process. If deployed across multiple backend instances, presence and rooms would need a shared adapter such as Redis.

Frontend Questions

How is frontend state managed?

The frontend uses JavaScript variables such as `currentWorkspace`, `workspaceBoards`, `activeBoardData`, `activityFeed`, `notificationFeed`, `workspaceMembers`, `boardFilters`, and `cardComments`. Persistent session and workspace ids are stored in `localStorage`.

What is `apiRequest`?

It is a shared helper that attaches JSON headers, adds the JWT Bearer token, calls the backend, parses JSON, and throws errors when the response is not ok.

How does the app know which backend URL to use?

It reads Vite placeholders for `VITE_API_URL` and `VITE_SOCKET_URL`, with local defaults like `http://localhost:5002/api` and `http://localhost:5002`.

How does the board render?

The frontend loads board data, sorts columns by position, filters cards if filters are active, maps each column into HTML, maps each card into card HTML, then injects it into the board container.

How are XSS risks reduced in rendering?

The frontend uses an `escapeHtml` helper before injecting user-controlled text into HTML templates in many render paths.

How do saved views work?

Saved views are frontend-side saved filter configurations stored in localStorage. They store board id, name, and filters such as assignee, priority, and due date.

Is saved views data stored on backend?

No, currently it is local to the browser. A future improvement would persist saved views per user in MongoDB.

How does the profile avatar work?

The frontend reads the image file with `FileReader`, stores it temporarily as base64, previews it, and sends it to the backend when saving the profile.

API Design Questions

Why REST API?

REST is simple, familiar, and maps well to resources such as users, workspaces, boards, cards, comments, activities, and notifications.

Give examples of REST endpoints.

- `POST /api/auth/register`
- `POST /api/auth/login`
- `GET /api/workspaces`
- `POST /api/workspaces`
- `GET /api/boards/:boardId`
- `POST /api/boards/:boardId/cards`
- `PATCH /api/boards/:boardId/cards/:cardId/move`

- `GET /api/notifications`

Why use PATCH for updates?

PATCH is appropriate for partial updates. For example, updating a card may only change title or assignee without replacing the entire card.

Why use separate controllers and routes?

Routes define URL structure and middleware. Controllers contain business logic. This keeps the backend organized.

What is middleware?

Middleware is a function that runs during request processing. In TeamOps, auth middleware verifies JWT and role middleware checks workspace permissions.

Tricky Examiner Questions

Your frontend package has React dependencies. Is this a React app?

No. Those dependencies are legacy from earlier setup, but the active frontend is not React. The actual source is standalone `index.html` plus `styles.css`. I would remove unused dependencies in cleanup, but I did not describe the app as React because that would be inaccurate.

Your landing page may mention PostgreSQL. Are you using PostgreSQL?

No. The active backend uses MongoDB and Mongoose. Any old text mentioning PostgreSQL is stale copy and should be updated. The real models and controllers use MongoDB.

Why is `db/init.sql` unused?

Because the project uses MongoDB, not PostgreSQL. MongoDB seeding is handled through `seedDatabase()` in the backend model file.

Why are all models in one file?

For this student project, keeping schemas and formatters together in `models/index.js` makes it easy to inspect the complete data model. In a larger production app, I would split models into separate files.

Why is the frontend in one big HTML file?

The repository architecture requires the active frontend to be standalone. It avoids a framework build step and keeps deployment simple. The tradeoff is that as the app grows, modularity becomes harder. A future improvement would be splitting JavaScript into ES modules while preserving the standalone architecture.

Is your app secure enough for production?

It has important security basics: bcrypt, JWT verification, email verification, RBAC, CORS config, role checks, data scoping, and Google token verification. For production hardening, I would add rate limiting, HTTP-only cookies, stronger token storage, hashed OTPs, better logging, automated tests, and a Redis adapter for scaled sockets.

Why not store JWT in cookies?

Cookies with `HttpOnly`, `Secure`, and `SameSite` flags are stronger against token theft through XSS. I used localStorage for simplicity in this lab project. If production security were the priority, I would move tokens to HTTP-only cookies and add CSRF considerations.

Can a user access a board by guessing its id?

They should not be able to access it unless they are a member of the workspace. The backend resolves the board's workspace and checks membership/role before returning data.

Can a viewer create a card using Postman?

No. Even if the frontend hides buttons, the backend route requires owner/admin/member role. A viewer request should be rejected.

What if two users move the same card at the same time?

The backend persists the latest valid move and recalculates positions. However, there is no conflict-resolution versioning yet. A future improvement would add optimistic concurrency, timestamps, or operation queues for stricter consistency.

What if Socket.io event is missed?

The database is the source of truth. The frontend refreshes board data after real-time events and also loads current board state through REST. If an event is missed, reloading the board retrieves the correct persisted data.

Why not use only Socket.io and no REST?

REST is better for standard CRUD and initial loading because it is request-response and easy to secure/debug. Socket.io is used for real-time notifications after changes.

Why not use only REST and no Socket.io?

Without Socket.io, users would need manual refresh or polling. Socket.io makes collaboration feel live and reduces unnecessary repeated polling.

What is the biggest technical challenge?

Keeping data scoped correctly across workspaces, boards, columns, cards, and roles while also broadcasting real-time updates. A card move is not just a UI action: it needs permission checks, board validation, position recalculation, activity logging, notification logic, and socket emission.

What is the biggest weakness?

The biggest weakness is lack of a full automated test suite and the large frontend file. The backend logic is meaningful enough that tests for auth, RBAC, invites, card movement, and notifications would add a lot of confidence.

How would you scale this?

I would use a managed MongoDB cluster, add indexes for common queries, move uploaded media to object storage, add Redis for Socket.io adapter and caching, run multiple backend instances behind a load balancer, use structured logging, and add monitoring.

How would you deploy it?

Backend as a web service with environment variables for MongoDB URI, JWT secret, CORS origin, frontend URL, and Google client id. Frontend as a static site with Vite build output and environment variables pointing to backend API and socket URL.

What would happen if `JWT_SECRET` changes?

Existing tokens signed with the old secret become invalid, so users need to log in again. That can be acceptable during rotation but should be planned.

What happens if email sending fails during registration?

The backend catches errors in registration generally and returns an error. For robust production behavior, email failure should be handled carefully, perhaps retrying or allowing resend verification.

Why are OTP tokens stored in database?

The backend needs to verify codes later and enforce expiry. Storing them in the database lets the server validate and delete them after use.

Should OTPs be hashed?

In a production system, yes. Plain OTP storage works functionally, but hashing OTPs would reduce risk if the database were leaked.

How do you prevent duplicate invite codes?

The backend generates a code and checks whether it already exists in the Workspace collection. It repeats until unique.

What if the invite token is expired?

The backend rejects it. Email invite tokens include `expiresAt` and a `used` flag.

Why are there both invite code and invite token flows?

Invite codes are simple and quick to share. Email invite tokens are more controlled because they are tied to a specific email, role, expiry, and used status.

How do you stop admins from removing owners?

The workspace controller checks target roles and requester roles. Admins can manage members/viewers but cannot manage owners.

What is a race condition in your project?

Two simultaneous card moves or role updates could overwrite each other without version checks. The current app handles normal usage, but strict concurrency could be improved with transactions or optimistic locking.

Do you use database transactions?

No, not currently. Some operations use `Promise.all`, but not MongoDB transactions. A production version could use transactions for operations like deleting boards with all related data.

What happens when a board is deleted?

The backend finds its columns and cards, deletes related comments, deletes cards, deletes columns, deletes activities for that board, deletes the board, and emits `board:deleted`.

What happens when a workspace is deleted?

The backend checks owner permission, deletes cards, activities, columns, boards, and then deletes the workspace.

What is denormalization?

Denormalization means storing some related or repeated data to reduce query complexity. TeamOps mostly uses references, but activity records include fields like workspaceId and boardId for easier querying and formatting.

How does `populate` work?

Mongoose `populate` replaces referenced object ids with selected fields from the referenced document, such as assignee name/email or comment user name.

Why are response keys sometimes snake_case?

The formatters intentionally return API-friendly JSON keys such as `workspace_id`, `column_id`, and `created_at`, while JavaScript model fields may use camelCase.

What is CORS and why configure it?

CORS controls which browser origins can call the backend. Since the frontend and backend may run on different ports/domains, the backend must allow the frontend origin.

Why does development allow localhost with any port?

That makes local testing easier when Vite or backend ports change. In production, origins should be restricted.

Deep Technical Questions

Explain `requireRole`.

`requireRole` is attached to protected routes after JWT auth. It resolves the workspace id from route params, request body, query, or related board/column/card id. Then it loads the workspace, checks if the

current user is in `workspace.members`, and verifies that their role is allowed for that route.

Explain `resolveBoardAccess`.

It receives user id, board id, and required role. It loads the board, obtains its workspace id, and calls workspace access checking. If allowed, it returns the board, workspace id, and member data.

Explain `formatCard`.

It converts a Mongoose card document into a clean API object with `id`, `column_id`, title, description, priority, `assignee_id`, formatted `due_date`, position, `created_at`, and `updated_at`.

Explain `createActivity`.

It creates an Activity document with user, workspace, board, and action. Then it formats the activity and emits `activity:new` to the board room.

Explain `createNotification`.

It creates a Notification document for a user, formats it, and emits `notification:new` to the private user room.

Explain comment tagging safety.

For board comments, the backend normalizes tagged card ids by checking that they belong to the current board. It normalizes tagged member ids by checking they belong to the current workspace. This prevents users from tagging arbitrary ids from outside the workspace.

Explain overdue card calculation.

Analytics loads cards, identifies done columns, and counts cards with due dates before today while excluding cards in done columns.

Explain completed-this-week calculation.

The backend finds done columns and counts cards whose current column is done and whose updated timestamp is after the start of the current week.

Explain `escapeRegex`.

It escapes special regex characters before building a case-insensitive duplicate column title check. Without it, a title containing regex characters could behave unexpectedly.

Explain `escapeHtml`.

It creates a temporary div, assigns text to `textContent`, and reads `innerHTML`. This converts potentially dangerous characters into safe HTML entities.

Explain localStorage usage.

The app stores token, user, active workspace id, active board id, invite code/link, and saved filter views. This keeps the session and active context after page refresh.

Coding and Implementation Questions

Which file starts the backend?

`teamops-backend/server.js`.

Which file has models?

`teamops-backend/models/index.js`.

Which file handles sockets?

`teamops-backend/socket/index.js`.

Which file handles auth middleware?

`teamops-backend/middleware/auth.js`.

Which frontend files matter?

`teamops-frontend/index.html` and `teamops-frontend/styles.css`.

How do you run backend?

```
cd teamops-backend
npm install
npm run dev
```

How do you run frontend?

```
cd teamops-frontend
npm install
npm run dev
```

What is the local backend URL?

The code defaults to `http://localhost:5002/api`, although the environment variable can override it.

What is the local frontend URL?

Vite usually serves it at `http://localhost:5173`.

Scenario Questions

A user says card movement works locally but not for teammates. What do you check?

I would check whether Socket.io is connected, whether both users joined the same `board:<boardId>` room, whether the backend emits `card:moved`, whether CORS/socket URL are correct, and whether the database update succeeds. If socket fails, refreshing should still show the persisted move.

A user can log in but sees no boards. What do you check?

I would check if the user belongs to any workspace, whether workspace id is stored correctly, whether `/api/workspaces` returns data, whether `ensureWorkspaceBoard` created a board, and whether the active board id in localStorage is valid.

A viewer says they cannot add a card. Is this a bug?

No. Viewers are read-only for cards. They can view board data but cannot create/edit cards.

A member says they cannot create columns. Is this a bug?

No. Columns change the board workflow, so only owners and admins can manage columns.

A notification does not appear instantly. What do you check?

Check whether the socket connected with a valid token, whether it joined the `user:<userId>` room, whether `createNotification` ran, and whether the frontend listens for `notification:new`. The notification should still be in the database and load from `/api/notifications`.

A card due date in the past is rejected. Why?

The backend prevents active cards from being scheduled in the past. Past due dates are allowed only for done cards or when preserving existing historical data.

Comparison Questions

TeamOps versus Trello?

Trello is a mature commercial product. TeamOps is a student-built implementation focused on core project management concepts: workspaces, boards, roles, cards, comments, activity, notifications, analytics, and real-time sync.

TeamOps versus Jira?

Jira is much more advanced for software teams, with workflows, issue types, sprints, reports, and integrations. TeamOps demonstrates a simplified version of collaborative task management with a cleaner academic scope.

Why not build an LLM chatbot like other students?

LLM projects are impressive, but TeamOps demonstrates full-stack product engineering fundamentals: authentication, authorization, data modeling, real-time systems, UI state, database consistency, and deployment. These are foundational skills behind real-world applications, including AI tools.

Can TeamOps integrate AI later?

Yes. Future AI features could include task summarization, auto-priority suggestions, sprint risk prediction, comment summaries, natural language card creation, or an assistant that answers workspace questions.

Examiner-Flattering Answers

Use these when you want to sound thoughtful.

What did you learn?

I learned that the difficult part of a full-stack app is not just making screens. The hard part is keeping frontend state, backend validation, database consistency, permissions, and real-time events aligned.

What would you improve first?

I would add automated tests for authentication, RBAC, invites, and card movement because those are the highest-risk flows. After that, I would modularize the frontend for maintainability.

What is your best design decision?

Using Socket.io rooms with REST APIs. REST gives a reliable source-of-truth API, while Socket.io adds live collaboration without replacing normal CRUD.

What is one tradeoff you made?

I kept the frontend standalone instead of using a framework. It simplified deployment and matched the project architecture, but it also makes the frontend file large. If the project grows, modularization would be important.

What is one production concern?

Scaling real-time presence and board rooms across multiple backend instances. That would require a shared Socket.io adapter such as Redis.

Rapid Fire Questions

Question	Short Answer
What is Node.js?	JavaScript runtime for server-side code
What is Express?	Web framework for Node APIs
What is MongoDB?	NoSQL document database
What is Mongoose?	ODM for MongoDB
What is JWT?	Signed token for stateless authentication
What is bcrypt?	Password hashing library
What is Socket.io?	Real-time bidirectional communication library
What is CORS?	Browser security policy for cross-origin requests
What is middleware?	Function that runs during request processing
What is REST?	Resource-based API design style

Question	Short Answer
What is CRUD?	Create, Read, Update, Delete
What is RBAC?	Role-based access control
What is `populate`?	Mongoose feature to load referenced documents
What is `lean()`?	Returns plain JS objects instead of Mongoose documents
What is a schema?	Structure definition for a document
What is an index?	Database structure for faster lookup
What is a room in Socket.io?	Group of sockets for targeted emission
What is localStorage?	Browser key-value storage
What is Vite?	Fast frontend dev/build tool
What is an environment variable?	Runtime configuration value

Hard Questions With Strong Answers

How do you ensure database consistency when moving cards?

The backend loads source and destination columns, verifies board ownership, inserts the card into the destination order, updates positions for destination cards, recalculates source column positions if needed, saves the moved card, and emits a real-time event. For stricter production consistency, I would wrap related updates in a MongoDB transaction.

How would you add tests?

I would start with backend integration tests using an isolated test database. Tests would cover registration, verification, login, workspace creation, role enforcement, card CRUD, card movement, comments, notifications, and invite flows. Then I would add Playwright tests for browser workflows.

How would you improve security?

I would add rate limiting, input validation with a schema library, HTTP-only cookie auth, hashed OTP tokens, stronger password policy, secure headers, centralized error handling, audit log immutability, and more restrictive production CORS.

How would you implement file attachments?

I would add an Attachment model with card id, uploaded by, filename, MIME type, size, storage URL, and created timestamp. Files would be uploaded to cloud object storage, not stored directly in MongoDB.

How would you add AI features?

I would add a backend endpoint that gathers scoped workspace data only after RBAC checks, then sends a summarized prompt to an AI model. Example features include sprint summary, risk detection,

auto-card generation, and comment summarization.

How would you prevent XSS?

Escape user-generated text before rendering, sanitize rich text if added, avoid dangerous `innerHTML` where possible, use Content Security Policy, avoid storing tokens in localStorage for production, and validate inputs on the backend.

How would you prevent NoSQL injection?

Avoid passing raw user objects directly into MongoDB query operators, validate and cast ids, use fixed query shapes, and sanitize inputs.

Why is backend validation more important than frontend validation?

Frontend validation improves UX, but attackers can bypass it. Backend validation protects the actual data and permissions.

How do you handle errors?

Controllers use try/catch and return appropriate status codes and messages. The frontend catches errors from `apiRequest`, shows toasts or inline messages, and avoids crashing the whole app.

What status codes do you use?

Common examples: `200` success, `201` created, `400` bad request, `401` unauthenticated, `403` unauthorized, `404` not found, `409` conflict, and `500` server error.

Data Flow Examples

Register and Verify Flow

```
User submits signup form
Frontend POST /api/auth/register
Backend hashes password
Backend creates unverified user
Backend creates OTP token
Backend sends email
User enters OTP
Frontend POST /api/auth/verify
Backend marks user verified
Backend returns JWT
Frontend saves session
```

Card Move Flow

```
User drags card
Frontend catches drop event
Frontend PATCH /api/boards/:boardId/cards/:cardId/move
Backend verifies JWT
Backend checks workspace role
Backend validates source/destination columns
Backend recalculates positions
Backend saves MongoDB updates
```

```
Backend creates activity
Backend emits card:moved
Other clients reload board data
```

Notification Flow

```
User assigns card to teammate
Backend creates card/update
Backend creates Notification document
Backend emits notification:new to user:<assigneeId>
Assignee frontend receives event
Frontend shows toast and reloads inbox
```

Diagrams to Draw in Viva

System Architecture

```
Frontend HTML/CSS/JS
     |
     | REST + JWT
     v
Express Controllers
     |
     v
Mongoose Models
     |
     v
MongoDB

Socket.io side channel:
Frontend <--> Socket.io rooms <--> Backend events
```

Entity Relationship

```
User -- member of --> Workspace -- has --> Board -- has --> Column -- has --> Card
 Card -- assigned to --> User
 Card -- has --> Comment
 Board -- has --> Comment
 Workspace/Board -- has --> Activity
 User -- has --> Notification
```

Things Not To Say

- Do not say it is a React app.
- Do not say the active database is PostgreSQL.
- Do not say Socket.io stores data. MongoDB stores data; Socket.io broadcasts events.
- Do not say frontend role checks are enough for security.
- Do not say JWT encrypts data. JWT is signed; it is not automatically encrypted.
- Do not say bcrypt encryption. Say bcrypt hashing.
- Do not say localStorage is the most secure token storage.

- Do not claim automated testing is complete if it is not.

Polished Answers for Weak Points

The frontend file is large. How do you defend it?

The project architecture intentionally uses a standalone frontend with no React source tree. I kept the implementation in `index.html` and `styles.css` to match that architecture and simplify deployment. I agree that modularizing JavaScript into separate ES modules would be a good next step as the project grows.

There are unused dependencies. How do you defend it?

Some dependencies appear to be legacy from previous setup. The real active frontend does not use React. Removing unused dependencies would be a cleanup task, but it does not change the working architecture.

There is no full test suite. How do you defend it?

The project currently focuses on implementing the full feature set. I still have basic verification commands, but a proper next step is backend integration tests and frontend end-to-end tests. I can clearly identify which flows need tests first: auth, RBAC, card movement, invites, and notifications.

Final Viva Closing Statement

TeamOps demonstrates that I understand more than UI design. I implemented a complete application flow from authentication and database modeling to REST APIs, role checks, real-time Socket.io updates, frontend rendering, notifications, activity tracking, and deployment configuration. The strongest part is the integration between backend persistence and live collaboration: every important board action is validated, saved, logged, and broadcast to the right users.